

Bitset

bitset

介绍

`std::bitset` 是标准库中的一个存储 0/1 的大小不可变容器。严格来讲，它并不属于 STL。

► bitset 与 STL

由于内存地址是按字节即 `byte` 寻址，而非比特 `bit`，一个 `bool` 类型的变量，虽然只能表示 0/1，但是也占了 1 `byte` 的内存。

`bitset` 就是通过固定的优化，使得一个字节的八个比特能分别储存 8 位的 0/1。

对于一个 4 字节的 `int` 变量，在只存 0/1 的意义下，`bitset` 占用空间只是其，计算一些信息时，所需时间也是其。

在某些情况下通过 `bitset` 可以优化程序的运行效率。至于其优化的是复杂度还是常数，要看计算复杂度的角度。一般 `bitset` 的复杂度有以下几种记法：（设原复杂度为）

1. ，这种记法认为 `bitset` 完全没有优化复杂度。
2. ，这种记法不太严谨（复杂度中不应出现常数），但体现了 `bitset` 能将所需时间优化至。
3. ，其中（计算机的位数），这种记法较为普遍接受。
4. ，其中为计算机一个整型变量的大小。

另外，`vector` 的一个特化 `vector<L Tbool>` 的储存方式同 `bitset` 一样，区别在于其支持动态开空间，`bitset` 则和我们一般的静态数组一样，是在编译时就开好了的。然而，`bitset` 有一些好用的库函数，不仅方便，而且有时可以实现 SIMD 进而减小常数。另外，`vector<L Tbool>` 的部分表现和 `vector` 不一致（如对 `std::vector<L Tbool> vec` 来说，`&vec[0] + i` 不等于 `&vec[i]`）。因此，一般不使用 `vector<L Tbool>`。

使用

参见 std::bitset - cppreference.com。

头文件

```
1 #include <L Tbitset>
```

指定大小

```
1 std::bitset<1000> bs; // a bitset with 1000 bits
```

构造函数

- `bitset()`: 每一位都是 `false`。
- `bitset(unsigned long val)`: 设为 `val` 的二进制形式。
- `bitset(const string& str)`: 设为 串 `str`。

运算符

- `operator []`: 访问其特定的一位。

- `operator ==/operator !=`: 比较两个 `bitset` 内容是否完全一样。
- `operator &/operator &=/operator |/operator |=/operator ^/operator ^=/operator ~`: 进行按位与/或/异或/取反操作。

注意: `bitset` 只能与 `bitset` 进行位运算, 若要和整型进行位运算, 要先将整型转换为 `bitset`。

- `operator <</operator >>/operator <<=/operator >>=`: 进行二进制左移/右移。

此外, `bitset` 还提供了 C++ 流式 IO 的支持, 这意味着你可以通过 `cin/cout` 进行输入输出。

成员函数

- `count()`: 返回 `true` 的数量。
- `size()`: 返回 `bitset` 的大小。
- `test(pos)`: 它和 `vector` 中的 `at()` 的作用是一样的, 和 `[]` 运算符的区别就是越界检查。
- `any()`: 若存在某一位是 `true` 则返回 `true`, 否则返回 `false`。
- `none()`: 若所有位都是 `false` 则返回 `true`, 否则返回 `false`。
- `all()`: 若所有位都是 `true` 则返回 `true`, 否则返回 `false`。
- 1. `set()`: 将整个 `bitset` 设置成 `true`。
- 2. `set(pos, val = true)`: 将某一位设置成 `true/false`。
- 1. `reset()`: 将整个 `bitset` 设置成 `false`。
- 2. `reset(pos)`: 将某一位设置成 `false`。相当于 `set(pos, false)`。
- 1. `flip()`: 翻转每一位。(, 相当于异或一个全是 `1` 的 `bitset`)
- 2. `flip(pos)`: 翻转某一位。
- `to_string()`: 返回转换成的字符串表达。
- `to_ulong()`: 返回转换成的 `unsigned long` 表达 (`long` 在 NT 及 32 位 POSIX 系统下与 `int` 一样, 在 64 位 POSIX 下与 `long long` 一样)。
- `to_ullong()`: (C++11 起) 返回转换成的 `unsigned long long` 表达。

另外, `libstdc++` 中有一些较为实用的内部成员函数¹:

- `_Find_first()`: 返回 `bitset` 第一个 `true` 的下标, 若没有 `true` 则返回 `bitset` 的大小。
- `_Find_next(pos)`: 返回 `pos` 后面 (下标严格大于 `pos` 的位置) 第一个 `true` 的下标, 若 `pos` 后面没有 `true` 则返回 `bitset` 的大小。

应用

[「LibreOJ β Round #2」贪心只能过样例](#)

这题可以用 `dp` 做, 转移方程很简单:

表示前 i 个数的平方和能否为 j , 那么 $f[i][j]$ (或起来)。

但如果直接做的话是 $O(n^2)$ 的, (看起来) 过不了。

发现可以用 `bitset` 优化, 左移再或起来就好了:

► 提交记录: [std::bitset](#)

由于 `libstdc++` 的实现为压 `__CHAR_BIT__ * sizeof(unsigned long)` 位的², 在一些平台中其为 `32`。所以, 可以手写 `bitset` (只需要支持左移后或起来这一种操作) 压 `__CHAR_BIT__ * sizeof(unsigned long long)` 位来进一步优化:

► 提交记录: [手写 bitset](#)

另外，加了几个剪枝的暴力也能过：

► 提交记录：[加了几个剪枝的暴力](#)

CF1097F Alex and a TV Show

题意

给你 n 个可重集，四种操作：

1. 把某个可重集设为一个数。
2. 把某个可重集设为另外两个可重集加起来。
3. 把某个可重集设为从另外两个可重集中各选一个数的 x 。即： x 。
4. 询问某个可重集中某个数的个数，在模 2 意义下。

可重集个数 n ，操作个数 m ，值域 V 。

做法

看到「在模 2 意义下」，可以想到用 `bitset` 维护每个可重集。

这样的话，操作 1 直接设，操作 2 就是异或（因为模 2 ），操作 3 就是直接查，但 4 操作 4 怎么办？

我们可以尝试维护每个可重集的所有约数构成的可重集，这样的话，操作 3 就是直接按位与。

我们可以把值域内每个数的约数构成的 `bitset` 预处理出来，这样操作 3 就解决了。操作 4 仍然是异或。

现在的问题是，如何通过一个可重集的约数构成的可重集得到该可重集中某个数的个数。

令原可重集为 S ，其约数构成的可重集为 T ，我们要求 T 中 x 的个数，用 [莫比乌斯反演](#) 推一推：

由于是模 2 意义下， x 和 1 是一样的，只用看 x 有没有平方因子即可。所以，可以对值域内每个数预处理出其倍数中除以它不含平方因子的位置构成的 `bitset`，求答案的时候先按位与再 `count()` 就好了。

这样的话，单次询问复杂度就是 $O(\sqrt{V})$ 。

至于预处理的部分，或者 T 预处理比较简单， S 预处理就如下面代码所示，复杂度为调和级数，所以是 $O(V \log V)$ 。

► 参考代码

与埃氏筛结合

由于 `bitset` 快速的连续读写效率，使得它非常适合用于与 [埃氏筛](#) 结合打质数表。

使用的方式也很简单，只需要将埃氏筛中的布尔数组替换成 `bitset` 即可。

► 速度测试

► 参考代码

与树分块结合

`bitset` 与树分块结合可以解决一类求树上多条路径信息并的问题，详见 [数据结构/树分块](#)。

与莫队结合

详见 [杂项/莫队配合 bitset](#)。

计算高维偏序

详见 [FHR 课件](#)。

参考资料与注释

1. [libstdc++: SGI STL extensions ↩](#)
 2. [libstdc++: std::bitset<_Nb> Class Template Reference ↩](#)
-

本页面最近更新：2024/10/9 22:38:42, [更新历史](#)

发现错误？想一起完善？ [在 GitHub 上编辑此页！](#)

本页面贡献者：[ouuan](#), [Xeonacid](#), [abc1763613206](#), [CCXXI](#), [Chrogeek](#), [cmpute](#), [countercurrent-time](#), [Enter-tainer](#), [Henry-ZHR](#), [i-Yirannn](#), [lr1d](#), [ksyx](#), [NachtgeistW](#), [sshwy](#), [StudyingFather](#), [Tiphereth-A](#), [yzy-1](#)

本页面的全部内容在 [CC BY-SA 4.0](#) 和 [SATA](#) 协议之条款下提供，附加条款亦可能应用